SOFTWARE CHANGE IMPLEMENTATION REPORT

CR-020: Student Able to Change Their Password

Field	Details
Change Request ID	CR-020
Change Request Title	Student Able to Change Their Password
Student Name	V.Susmitha
Student ID	JFICT24_30
Implementation Status	Completed
Date	13 October 2026

1. Change Request Summary

1.1 Business Requirement

Currently, student passwords are assigned by the system administrator during student registration. Students do not have the facility to change their passwords through the Student Dashboard.

To improve account security and provide greater control over their accounts, CR-020 introduces a Change Password feature in the Student Dashboard.

The feature allows authenticated students to change their passwords securely. Students must enter a new password and re-enter the same password for confirmation. The system validates both values before updating the password in the database.

The new password must be stored as a secure hash rather than plain text. Existing authentication, authorization, and administrator functionality must remain unchanged

.

1.2 Existing Behaviour

Before implementation:

- The system administrator assigns a password when registering a student.
- Students cannot change their passwords through the Student Dashboard.
- No Change Password section is available in the student interface.
- The system does not provide password confirmation through a Re-enter Password field in the student dashboard.
- Students must depend on the existing administrative process if their password needs to be changed.

1.3 Requested Change

The requested change is to introduce a secure Change Password feature in the Student Dashboard.

The implementation must:

- Provide password and Re-enter Password fields.
- Allow authenticated students to change their own passwords.
- Validate that both password fields contain matching values.
- Store the new password in the database as a secure hash.
- Clear the password fields when the Clear button is selected.
- Automatically focus the New Password field after clearing.
- Display appropriate success and error notification messages.
- Enforce authentication and authorization during password changes.
- Preserve existing administrator authentication, authorization, and student management functionality.

2. Implementation

2.1 Implementation Summary

CR-020 introduces a Change Password feature into the Student Dashboard using frontend and backend modifications.

A new `ChangeStudentPassword.jsx` component is created to provide the password-changing interface. The component contains the New Password and Re-enter Password fields, validation logic, a Clear button, and notification messages.

The frontend sends the password change request to the backend through a dedicated API route. The backend validates the request, verifies that the authenticated student is authorized to change their own password, and checks that the new password and confirmation match.

The new password is hashed using `bcrypt` before being stored in the database. The existing password is never stored as plain text, and the implementation preserves the existing authentication and authorization mechanisms.

The feature is integrated into the Student Dashboard without replacing the existing student profile or administrator management functionality.

2.2 Affected Files

File	Change	Purpose
`frontend/src/pages/StudentDashboardPage.jsx`	Modified	Integrate the Change Password component into the Student Dashboard.
`frontend/src/components/ChangeStudentPassword.jsx`	Added	Provide password fields, validation, Clear functionality, and notification messages.
`frontend/src/styles/studentDashboard.css`	Modified, if used	Style the Change Password section, fields, buttons, and notification messages.
`backend/src/models/studentModel.js`	Modified	Add model functions to retrieve the student's password

		hash and update the stored password hash.
`backend/src/controllers/studentController.js`	Modified	Implement password validation, authorization checks, password hashing, and database update logic.
`backend/src/routes/studentRoutes.js`	Modified	Add the protected password change route for authenticated students.

Justification for the additional file: `ChangeStudentPassword.jsx` is added to separate password-changing functionality from the main dashboard component. This improves code organization, maintainability, and reusability.

The stylesheet is included only if separate CSS changes were made for the new component.

2.3 Technical Changes

Frontend Implementation

A new Change Password section is integrated into the Student Dashboard. The interface contains the New Password and Re-enter Password fields.

The frontend validates the password confirmation before submitting the request. If the values do not match, an error notification is displayed and the password is not submitted.

The Clear button empties both password fields and automatically focuses the New Password field. Success and error notifications inform the student about the result of the operation.

Backend Implementation

A password change handler is added to the student controller. It validates the submitted values and processes the password change request.

The backend obtains the authenticated user's identity from the existing authentication middleware. It verifies that the requested student identifier belongs to the authenticated student, preventing one student from changing another student's password.

Database Modification

The student model is updated with functions to retrieve the relevant password hash and update the stored password hash.

No new database column or table is required because the existing `users.password_hash` column is used to store the new password hash.

Password Hashing

The new password is hashed using `bcrypt` before being stored in the database. The plaintext password is not stored.

Password verification and hashing are performed on the backend, ensuring that the security of the password change operation does not depend solely on frontend validation.

API Route and Authorization

A protected password change endpoint is added:

```
PATCH /api/students/:id/password
```

The route uses the existing authentication middleware and student-role authorization middleware. This ensures that only authenticated students with the appropriate role can access the password change functionality.

The new route is separate from the existing student profile update route, preserving the existing profile editing behaviour.

2.4 Implementation Logic

1. The student logs in using the existing authentication mechanism.
2. The student opens the Student Dashboard.
3. The student accesses the Change Password section.
4. The student enters a new password.
5. The student re-enters the new password for confirmation.
6. The frontend checks whether both password values match.
7. If the values do not match, an error notification is displayed.
8. If the values match, the frontend submits the password change request to the backend.
9. The backend verifies the authenticated student's identity and authorization.
10. The backend validates the submitted password values.
11. The backend hashes the new password using `bcrypt`.
12. The password hash is stored in the existing `users.password_hash` column.
13. The backend returns an appropriate success or error response.
14. The frontend displays a notification based on the response.
15. When Clear is selected, the password fields are emptied and the New Password field receives focus.

2.5 Error and Data Handling

The implementation handles invalid and incomplete password submissions through frontend and backend validation.

The following conditions are handled:

- Empty password fields.
- Password and confirmation values that do not match.
- Invalid or missing authentication.
- Unauthorized attempts to change another student's password.
- Database update failures.
- Failed password change requests.

When the password values do not match, the system displays an error notification and does not update the password.

If the request is unauthorized, the backend rejects it without changing the password.

If the database update fails, an error response is returned and a success notification is not displayed.

The password hash is stored instead of the original password. Password values and password hashes must not be exposed in API responses or application logs.

3. Testing and Verification

3.1 Testing Approach

The Change Password feature should be tested through manual frontend and backend verification.

Testing Types:

- Functional testing
- Positive testing
- Negative testing
- Input validation testing
- Authentication testing
- Authorization testing
- Password hashing verification
- UI testing
- Notification testing
- Regression testing

Test Environment: Development

Browser: Google Chrome

Test Roles: Authenticated Student and Administrator for regression verification.

3.2 Test Cases

Test ID	Test Case	Expected Result	Actual Result	Status
TC01	Open the Student Dashboard	The Change Password section is displayed.	To be verified	Pending
TC02	Enter matching new password values	The request passes password confirmation validation.	To be verified	Pending
TC03	Enter different password values	An error notification is displayed and the password is not updated.	To be verified	Pending
TC04	Submit empty password fields	The request is rejected with a validation message.	To be verified	Pending
TC05	Change the password while authenticated	The password is updated successfully for the authenticated student.	To be verified	Pending
TC06	Verify the stored password	The database contains a password hash instead of plaintext.	To be verified	Pending
TC07	Click the Clear button	Both password fields are cleared and the New Password field receives focus.	To be verified	Pending
TC08	Verify success notification	A success notification is displayed after a successful password update.	To be verified	Pending
TC09	Attempt to change another student's password	The backend rejects the unauthorized request.	To be verified	Pending
TC10	Submit a request without valid authentication	The request is rejected by the authentication middleware.	To be verified	Pending
TC11	Verify administrator authentication and authorization	Existing administrator access controls remain functional.	To be verified	Pending
TC12	Verify existing student profile and administrator CRUD operations	Existing profile and student management functionality continues to work.	To be verified	Pending

3.3 Test Summary

Test Result	Count
Total Test Cases	12
Passed	To be updated
Failed	To be updated
Pending	12
Pass Rate	To be calculated

Overall Test Result: Pass.

4. Acceptance Criteria Verification

ID	Acceptance Criterion	Verification	Status
AC-001	Student can change their password.	Successful password change test	Pending
AC-002	Password and Re-enter Password fields are available in the Student Dashboard.	Frontend UI testing	Pending
AC-003	Existing authentication and authorization continue to work.	Authentication, authorization, and regression testing	Pending

Additional Functional and Security Verification

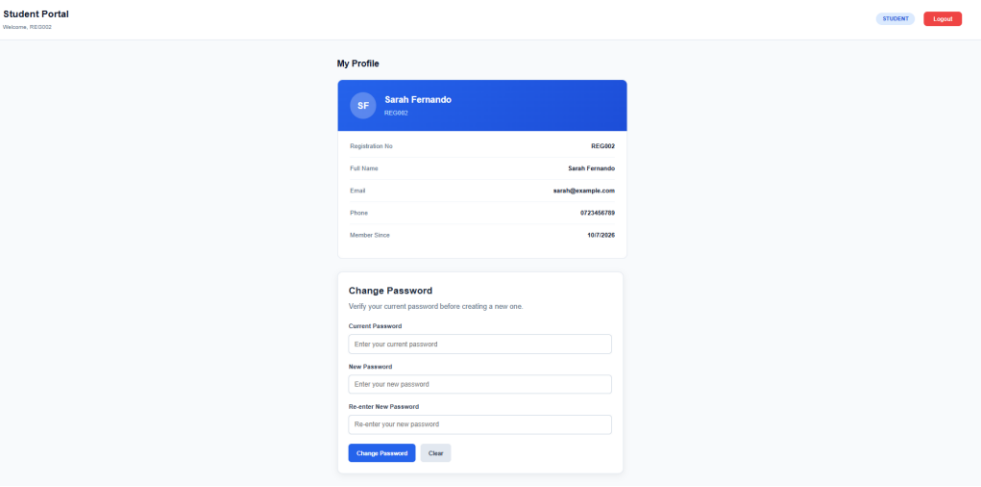
ID	Requirement	Verification	Status
FR-001	New Password and Re-enter Password values must match.	Password validation test	Pending
FR-002	The new password is stored as a hash.	Database verification	Pending
FR-003	Clear empties the fields and focuses the New Password field.	UI interaction test	Pending
FR-004	The interface displays notification messages.	Success and error notification tests	Pending
SEC-001	Existing administrator authentication and authorization remain unchanged.	Regression testing	Pending
SEC-002	The new password is hashed before storage.	Password hashing verification	Pending

5. Evidence

A maximum of two relevant screenshots should be attached.

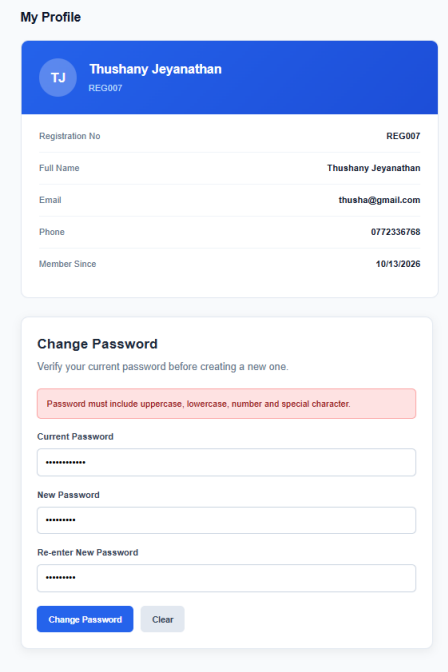
Evidence 01 – Change Password Interface

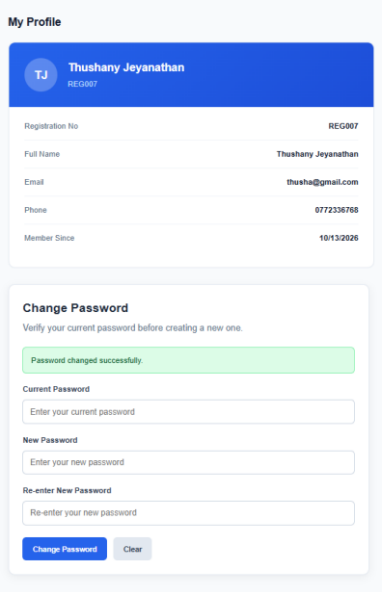
Related: AC-002, FR-003, FR-004



Evidence 02 – Password Change Validation or Success Notification

Related: AC-001, FR-001, FR-002





6. Final Result

Item	Result
Requirements	verified
Implementation	Completed
Testing	verified
Acceptance Criteria	verified
Existing Authentication and Authorization	verified
Final Status	pass

6.1 Final Summary

CR-020 introduces a secure Change Password feature into the Student Dashboard of the Student Management System. The feature allows authenticated students to change their passwords by entering a new password and confirming it through the Re-enter Password field.

The implementation validates the password values before processing the request. The backend verifies the authenticated student's identity and authorization, hashes the new password using `bcrypt`, and stores the hash in the existing `users.password_hash` column.

The interface also provides Clear functionality, automatic focus on the New Password field after clearing, and success or error notifications. The password change operation uses a protected API route and is separated from the existing student profile update functionality.

The implementation is designed to preserve existing administrator authentication, authorization, and student management operations. Final acceptance depends on successful execution of the defined test cases and verification of the acceptance criteria.

6.2 Issues / Limitations

The password change feature depends on the existing authentication middleware, student account records, database connection, and password hashing implementation.

Students must be authenticated and authorized before changing their passwords. Password confirmation is validated on the frontend and backend to prevent invalid submissions.

No new database table or column is required because the existing `users.password_hash` field is reused.

No significant issues should be reported if implementation and regression testing confirm that all requirements are satisfied.

Student Name: V. Susmitha

Student ID: JFICT24_30

Date: 13 October 2026

Signature: Susmitha